PEMROGRAMAN BERORIENTASI OBJEK 2026

Abstract Class dan *Interface*

Kerangka yang sengaja belum lengkap dan kontrak kemampuan: abstraksi, pilar keempat PBO

PERTEMUAN

10

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Menjelaskan arti **abstraksi** dengan analogi sehari-hari.
2. Menjelaskan ciri **kelas abstrak** dan *method* abstrak, termasuk kapan pelanggarannya ketahuan di Python dan di Java.
3. Menjelaskan pola ***template method***: *method* konkret yang memanggil *method* abstrak.
4. Menjelaskan ***interface*** sebagai kontrak kemampuan dan mengapa satu kelas boleh memenuhi banyak *interface*.
5. Memilih antara kelas abstrak dan *interface* untuk sebuah kasus, lengkap dengan alasannya.
6. Membaca dan menggambar kelas abstrak, *interface*, dan relasi realisasi pada diagram kelas UML.

Mengingat Kembali



Dari pertemuan sebelumnya

- **Polimorfisme**: satu panggilan *method*, perilaku berbeda tergantung objek sebenarnya. Kuncinya *override*.
- Parameter bertipe **kelas induk** boleh menunjuk objek **kelas anak**, misalnya `Pesanan.bayar(pembayaran)`.
- Python mengenal *duck typing*, sedangkan Java mewajibkan **tipe bersama**.
- Yang janggal: objek `Pembayaran` sendiri bisa dibuat, dan nama metodenya “Pembayaran umum”.

Agenda Hari Ini

1. Masalah “pembayaran umum” dan abstraksi
2. Kelas abstrak dan *method* abstrak
3. *Template method*: kerangka tetap
4. *Interface*: kontrak kemampuan
5. Kelas abstrak atau *interface*? Plus UML
6. Studi kasus KedaiKita

BAGIAN 1

1

Abstraksi: Menonjolkan yang Penting

Dari kasir yang kebingungan menuju ide besar pilar keempat PBO.

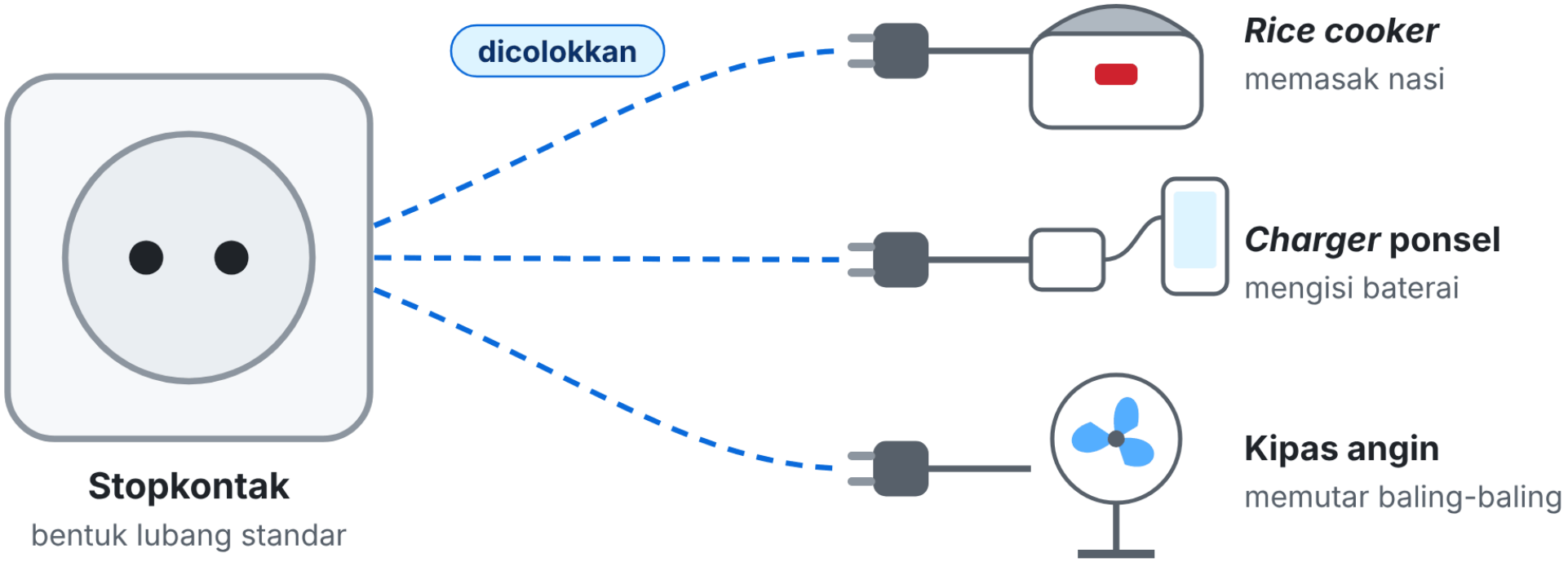
Masalah: “Pembayaran Umum” yang Janggal

- Pelanggan berkata, “Saya bayar pakai pembayaran umum.” Kasir bingung: yang ada selalu jenis tertentu.
- **Masalah 1:** objek `Pembayaran` “umum” bisa dibuat, padahal wujudnya tidak jelas.
- **Masalah 2:** kelas anak baru, misalnya `PembayaranEWallet`, **lupa** meng-*override* `proses()`.
- Akibatnya, `proses()` milik induk yang selalu berhasil dipakai: pesanan tercatat **LUNAS** tanpa uang masuk!

Perhatikan

Masalah kedua paling berbahaya karena program tidak protes sama sekali. Kita baru sadar setelah uang di laci kasir tidak cocok dengan laporan.

Stopkontak: Satu Standar untuk Banyak Alat



Stopkontak hanya peduli bentuk colokan, bukan alatnya. Pemakai cukup mengenal kontraknya, tanpa perlu tahu detail di dalam setiap alat.

Abstraksi: Apa, Bukan Bagaimana

Abstraksi (*abstraction*)

Cara berpikir yang **menonjolkan hal yang penting** (apa yang bisa dilakukan) dan **menyembunyikan detail** yang tidak perlu diketahui pemakai (bagaimana cara melakukannya).

- Kasir KedaiKita cukup berkata “proses pembayaran ini”, tanpa peduli tunai, QRIS, atau kartu.
- **Kelas abstrak**: kerangka bersama untuk kelas-kelas yang **bersaudara** (hubungan “adalah”).
- ***Interface***: kontrak kemampuan untuk kelas yang **belum tentu bersaudara** (hubungan “bisa”).

BAGIAN 2

2 Kelas Abstrak dan *Template Method*

Cetak biru yang sengaja belum lengkap, janji yang wajib ditepati, dan kerangka yang tetap.

Kelas Abstrak: Cetak Biru yang Sengaja Belum Lengkap

Kelas abstrak (*abstract class*)

Kelas yang **tidak bisa dibuat objeknya secara langsung**. Kelas ini menjadi kerangka bersama bagi kelas-kelas anak. Lawannya adalah **kelas konkret**: kelas biasa yang boleh dibuat objeknya.

- Di diler, kamu tidak bisa membeli “kendaraan” saja; yang ada selalu motor atau mobil.
- *Method* abstrak hanya punya “kepala” tanpa isi. Kelas anak konkret **wajib** meng-*override*-nya.
- Tetap boleh punya atribut, konstruktor, dan *method* konkret, serta boleh dipakai sebagai tipe.

Menandai Kelas dan *Method* Abstrak

Python: mewarisi `ABC` dan memakai `@abstractmethod`. Java: kata kunci `abstract`. `PersegiPanjang` mengisinya, jadi ia konkret.

Python python/contoh/contoh02_kelas_abstrak.py

```python
from abc import ABC, abstractmethod


class Bentuk(ABC):
    # ... (dilewati)
    @abstractmethod
    def luas(self) -> int:
        """Menghitung luas (cm2)."""
    # ... (dilewati)

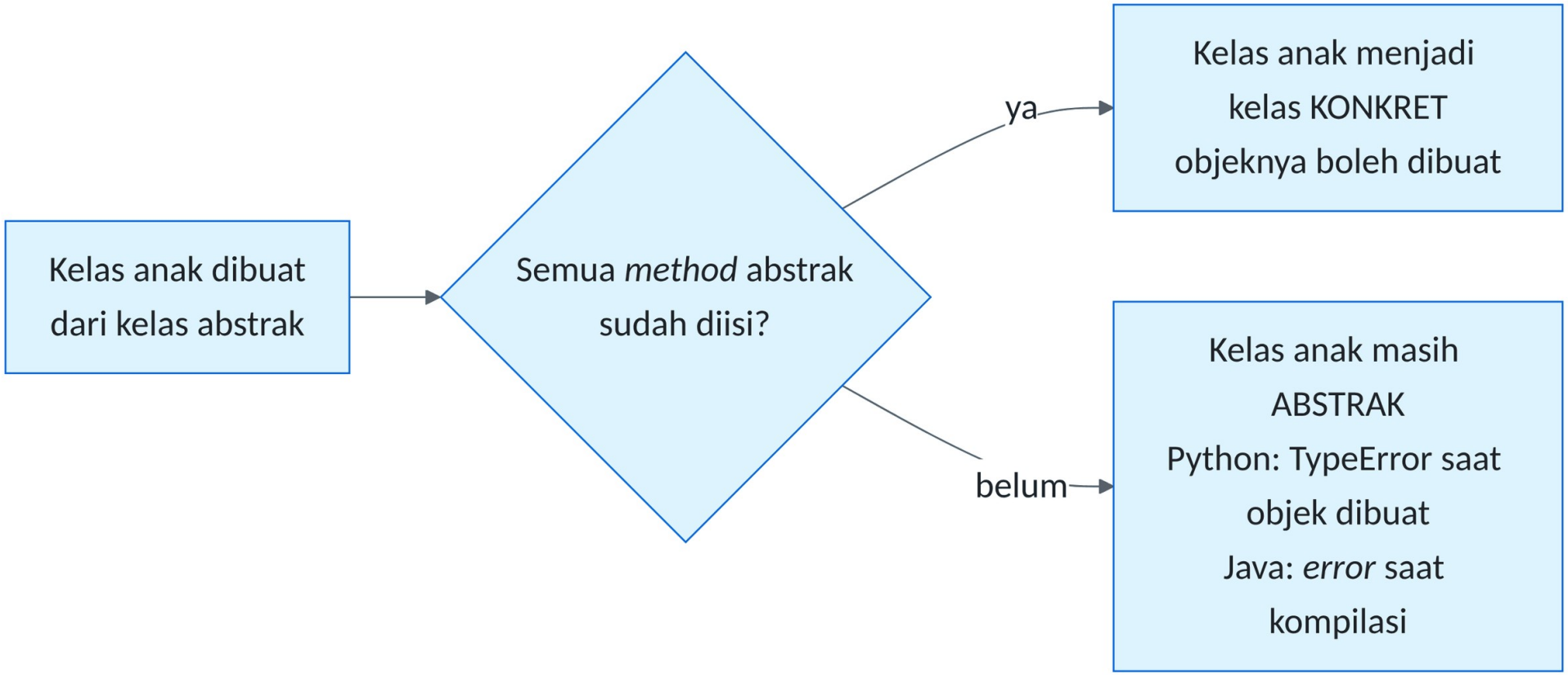
class PersegiPanjang(Bentuk):
    # ... (dilewati)
    def luas(self) -> int:
        return self.panjang * self.lebar
```

Java java/contoh/Contoh02KelasAbstrak.java

```java
abstract class Bentuk {
    /** Menghitung luas (cm2). */
    public abstract int luas();
    // ... (dilewati)
}

class PersegiPanjang extends Bentuk {
    // ... (dilewati)
    @Override
    public int luas() {
        return panjang * lebar;
    }
    // ... (dilewati)
}
```

Method Abstrak: Janji yang Wajib Ditepati



Override biasa itu **boleh**, sedangkan *override method* abstrak itu **wajib**. Kelas abstrak mengubah "sebaiknya" menjadi "harus".

Template Method: Kerangka Tetap, Detail Menyusul

`tampilkan()` menetapkan urutan langkah. `judul()` dan `isi()` diisi kelas anak; `penutup()` adalah *hook* yang boleh diganti.

Python python/contoh/contoh04_template_method.py

```python
def tampilkan(self):
    print("=" * LEBAR)
    print(self.judul())
    print("-" * LEBAR)
    for baris in self.isi():
        print(baris)
    print("-" * LEBAR)
    print(self.penutup())
    print("=" * LEBAR)

@abstractmethod
def judul(self) -> str:
    """Judul laporan."""
```

Java java/contoh/Contoh04TemplateMethod.java

```java
public final void tampilkan() {
    System.out.println("=".repeat(LEBAR));
    System.out.println(judul());
    System.out.println("-".repeat(LEBAR));
    for (String baris : isi()) {
        System.out.println(baris);
    }
    System.out.println("-".repeat(LEBAR));
    System.out.println(penutup());
    System.out.println("=".repeat(LEBAR));
}

/** Judul laporan. */
public abstract String judul();
```

BAGIAN 3

3

Interface: Kontrak Kemampuan

Untuk kelas-kelas yang tidak bersaudara, tetapi sama-sama bisa melakukan sesuatu.

Interface: Tanda Kemampuan, Bukan Keturunan

Interface (antarmuka)

Kontrak yang berisi daftar kemampuan (*method*) yang **wajib** dimiliki oleh kelas yang menyatakan diri memenuhinya. *Interface* tidak peduli kelas itu keturunan siapa. Hubungannya "bisa" (*can-do*), bukan "adalah" (*is-a*).

- Seperti **SIM C**: tanda **kemampuan** mengendarai motor, bukan tanda keturunan.
- Tidak bisa dibuat objeknya. Isinya terutama *method* tanpa isi, tanpa atribut biasa.
- Python: kelas ABC yang semua *method*-nya abstrak. Java: kata kunci `interface` dan `implements`.

Kontrak `BisaDinyalakan` dan Kelas yang Memenuhinya

Python mewarisi kelas `ABC` yang berisi *method* abstrak saja. Java menulis `interface`, lalu kelasnya menulis `implements`.

Python python/contoh/contoh05_interface.py

```python
class BisaDinyalakan(ABC):
    """Kontrak: benda ini bisa dinyalakan dan dimatikan."""

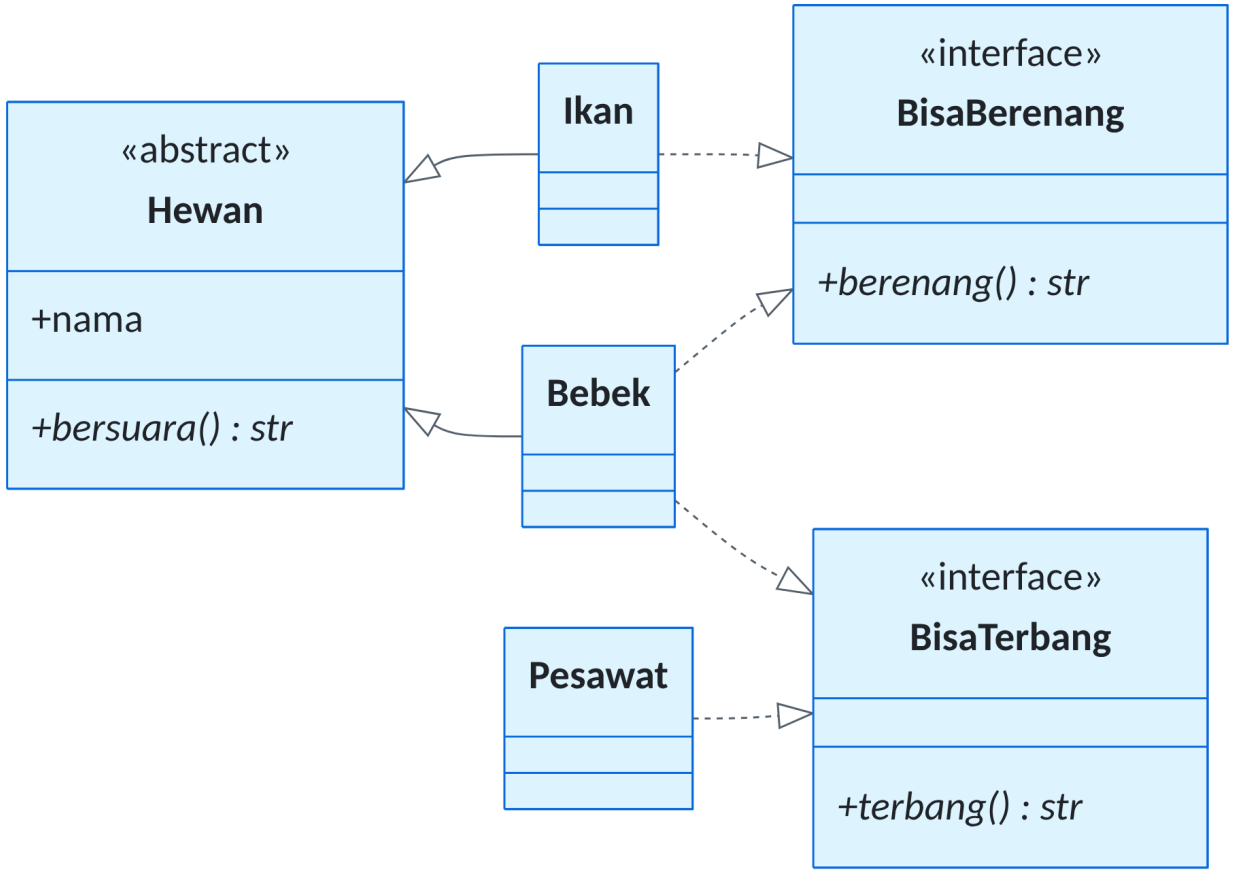
    @abstractmethod
    def nyalakan(self):
        """Menyalakan benda."""
    # ... (dilewati)

class Lampu(BisaDinyalakan):
    # ... (dilewati)
    def nyalakan(self):
        self._menyala = True
        print(f"Lampu {self._ruangan} menyala")
    # ... (dilewati)
```

Java java/contoh/Contoh05Interface.java

```java
interface BisaDinyalakan {
    void nyalakan();
    void matikan();
    boolean sedangMenyala();
}
class Lampu implements BisaDinyalakan {
    // ... (dilewati)
    @Override
    public void nyalakan() {
        menyala = true;
        System.out.println("Lampu " + ruangan + " menyala");
    }
    // ... (dilewati)
}
```

Satu Kelas, Banyak *Interface*



`Bebek` punya satu induk dan dua *interface*. `Pesawat` bukan `Hewan`, tetapi tetap `BisaTerbang`.

Fitur Khusus Tiap Bahasa

Fitur	Bahasa	Arti singkat
Default method	Java (sejak Java 8)	*Method* **berisi** di dalam *interface*; otomatis didapat, boleh di-*override*
Method `static` dan `private` di *interface*	Java (sejak Java 8 dan 9)	*Method* pembantu milik *interface* itu; tidak diwariskan
Konstanta di *interface*	Java	Nilai tetap yang menempel pada *interface*, misalnya `BINTANG_MAKS = 5`
`typing.Protocol`	Python	Kontrak **struktural**: cukup punya *method* yang cocok, tanpa mewarisi

`Protocol` diperiksa alat pemeriksa tipe di editor, misalnya Pylance atau mypy, bukan oleh Python saat program berjalan.

BAGIAN 4

4

Kelas Abstrak atau *Interface*?

Panduan memilih, lalu cara menggambar keduanya di diagram kelas UML.

Memilih: Kelas Abstrak atau *Interface*

Aspek	Kelas abstrak	*Interface*
Pertanyaan kunci	"X **adalah** sejenis Y?" (*is-a*)	"X **bisa** melakukan Y?" (*can-do*)
Hubungan antarkelas	Kelas-kelas yang bersaudara	Kelas apa saja, tidak harus bersaudara
Atribut dan konstruktor	Boleh	Tidak (Java hanya boleh konstanta)
Method berisi	Boleh	Java: lewat *default method*; Python: boleh, tetapi sebaiknya hemat
Satu kelas boleh memakai	Java: satu (`extends`); Python: boleh lebih	Banyak (`implements A, B`)
Contoh di KedaiKita	`Pembayaran`	`DapatDicetak`

Keduanya tidak bisa dibuat objeknya. Satu kelas boleh memakai keduanya sekaligus, seperti Bebek.

Abstraksi di Diagram Kelas UML

Elemen	Notasi UML (di kertas)	Di Mermaid
Kelas abstrak	Nama kelas ditulis *miring*, atau diberi label `<<abstract>>`	Label `<<abstract>>`
Method abstrak	Nama *method* ditulis *miring*	Tanda `*` di akhir, misalnya `+luas() int*`
Interface	Label `<<interface>>` di atas nama	Label `<<interface>>`
Pewarisan (*extends*)	Garis **lurus** + segitiga kosong menunjuk induk	`Induk <|-- Anak`
Realisasi (*implements*)	Garis **putus-putus** + segitiga kosong menunjuk *interface*	`Kelas ..|> Interface`

Mermaid tidak bisa memiringkan nama kelas, jadi kelas abstrak di materi ini ditandai label `<<abstract>>`.

STUDI KASUS

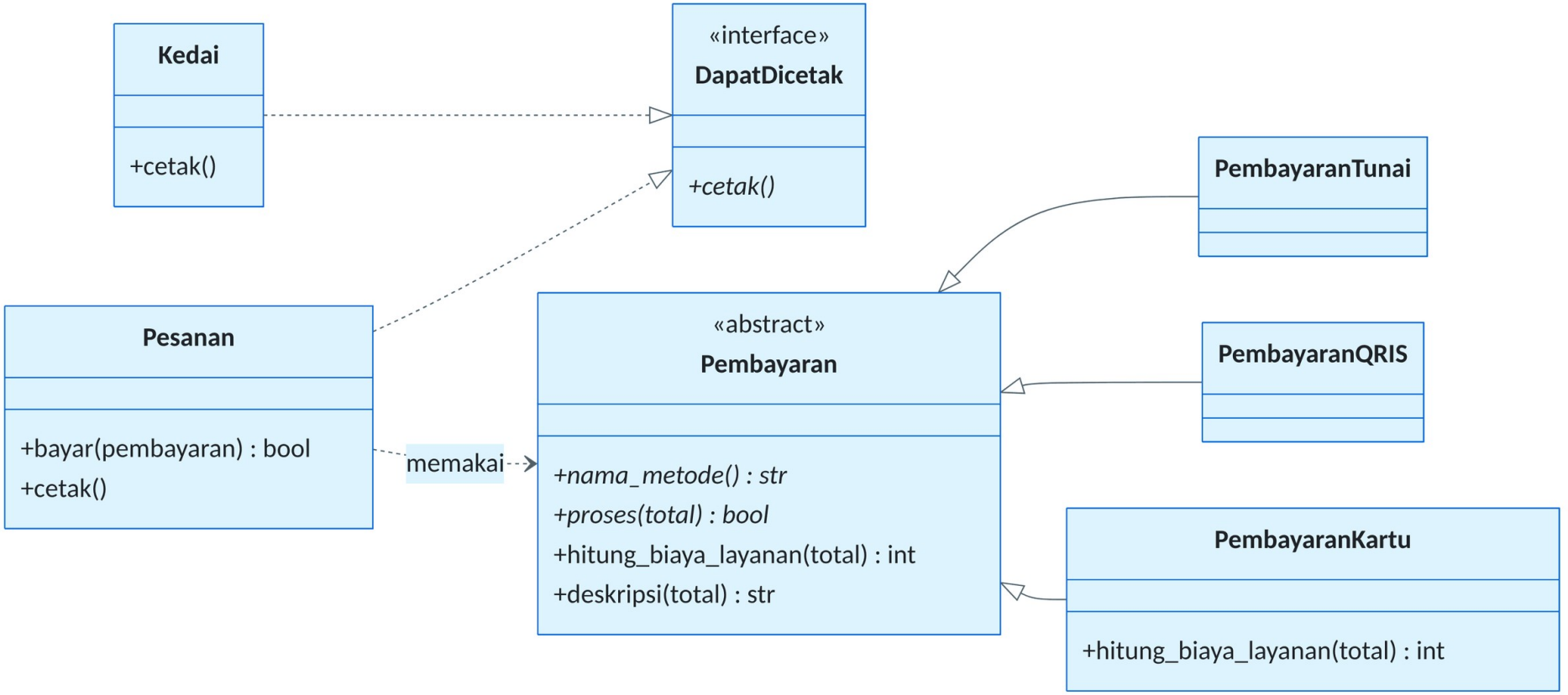


Studi Kasus KedaiKita

`Pembayaran` menjadi kelas abstrak, sedangkan `Kedai` dan `Pesanan` memenuhi kontrak `DapatDicetak`.

Rancangan: Pembayaran Abstrak dan Kontrak `DapatDicetak`

Studi kasus KedaiKita



Kotak `PembayaranTunai` dan `PembayaranQRIS` tampak kosong karena isinya tetap seperti di Pertemuan 09: keduanya sudah mengisi kedua *method* abstrak itu.

Pembayaran Kini Kelas Abstrak

Studi kasus KedaiKita

Nama metode dan `proses()` kini **wajib** diisi setiap jenis pembayaran. `deskripsi()` yang konkret memakai keduanya.

Python python/studi-kasus/pembayaran.py

```python
class Pembayaran(ABC):
    # ... (dilewati)
    @abstractmethod
    def nama_metode(self) -> str:
        """Nama metode pembayaran, misalnya 'Tunai'."""

    @abstractmethod
    def proses(self, total: int) -> bool:
        """Memproses pembayaran. ..."""

    def hitung_biaya_layanan(self, total: int) -> int:
        # ... (dilewati)
        return 0

    def deskripsi(self, total: int) -> str:
        biaya = format_rupiah(self.hitung_biaya_layanan(total))
        return f"{self.nama_metode()} (biaya layanan {biaya})"
```

Java java/studi-kasus/Pembayaran.java

```java
public abstract class Pembayaran {

    /** Nama metode pembayaran, misalnya "Tunai". */
    public abstract String namaMetode();
    // ... (dilewati)
    public abstract boolean proses(int total);
    // ... (dilewati)
    public int hitungBiayaLayanan(int total) {
        return 0;
    }
    // ... (dilewati)
    public String deskripsi(int total) {
        String biaya = Rupiah.format(hitungBiayaLayanan(total));
        return namaMetode() + " (biaya layanan " + biaya + ")";
    }
}
```

`Kedai` Memenuhi Kontrak `DapatDicetak`

Studi kasus KedaiKita

Kontrak menuntut nama `cetak()`, jadi `tampilkan_menu()` berganti nama. Di `Pesanan`, `cetak_struk()` juga menjadi `cetak()`.

Python python/studi-kasus/kedai.py

```python
from dapat_dicetak import DapatDicetak
# ... (dilewati)
class Kedai(DapatDicetak):
    """Kedai yang punya daftar menu dan catatan pesanan."""
    # ... (dilewati)
    def cetak(self):
        # ... (dilewati)
        print(f"=== MENU {self._nama} ===")
        for menu in self._daftar_menu:
            print(menu)
```

Java java/studi-kasus/Kedai.java

```java
/** Kedai yang punya daftar menu dan catatan pesanan. */
public class Kedai implements DapatDicetak {
    // ... (dilewati)
    @Override
    public void cetak() {
        System.out.println("=== MENU " + nama + " ===");
        for (Menu menu : daftarMenu) {
            System.out.println(menu);
        }
    }
}
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
“Kelas abstrak tidak boleh punya atribut dan konstruktor.”	Boleh. Kelas abstrak justru tempat yang tepat untuk data dan kode bersama. Konstruktornya dipanggil saat objek kelas anak dibuat.
“Kelas abstrak tidak bisa dibuat objeknya, jadi tidak bisa dipakai sebagai tipe.”	Variabel, parameter, dan *list* bertipe kelas abstrak atau *interface* sangat umum, misalnya parameter `pembayaran` di `Pesanan.bayar()`. Isinya selalu objek kelas konkret.
“*Interface* sama saja dengan kelas abstrak, hanya beda nama.”	Tujuannya berbeda. Kelas abstrak menyatukan kelas yang **bersaudara** beserta datanya; *interface* menyatukan kelas yang **belum tentu bersaudara** berdasarkan kemampuannya.

Kuis Cepat

Method konkret di kelas abstrak yang menetapkan **urutan langkah** sebuah proses, lalu memanggil *method* abstrak untuk langkah-langkah yang berbeda di setiap kelas anak, disebut …

A. Konstruktor

B. *Default method*

C. *Template method*

D. *Getter*

Kuis Cepat: Jawaban

Method konkret di kelas abstrak yang menetapkan **urutan langkah** sebuah proses, lalu memanggil *method* abstrak untuk langkah-langkah yang berbeda di setiap kelas anak, disebut ...

A Konstruktor

B *Default method*

✓ *Template method*

D *Getter*

Jawaban: C

Template method, misalnya `tampilkan()` pada `Laporan` atau `deskripsi()` pada `Pembayaran`, menetapkan kerangka yang sama bagi semua kelas anak. *Default method* (B) adalah *method* berisi di *interface* Java; ia bisa memanggil *method* abstrak, tetapi bukan nama pola ini. Konstruktor (A) dan *getter* (D) tidak berhubungan dengan urutan langkah.

Kuis Cepat

Situasi manakah yang **paling tepat** diselesaikan dengan *interface*?

A `Makanan` dan `Minuman` sama-sama menu yang punya kode, nama, dan harga

B `Kedai` dan `Pesanan` tidak bersaudara, tetapi sama-sama perlu dicetak oleh satu fungsi yang sama

C Membuat banyak objek menu dari satu kelas `Menu`

D Menyembunyikan atribut harga agar tidak diubah sembarangan dari luar kelas

Kuis Cepat: Jawaban

Situasi manakah yang **paling tepat** diselesaikan dengan *interface*?

A — `Makanan` dan `Minuman` sama-sama menu yang punya kode, nama, dan harga

✓ **B** — `Kedai` dan `Pesanan` tidak bersaudara, tetapi sama-sama perlu dicetak oleh satu fungsi yang sama

C — Membuat banyak objek menu dari satu kelas `Menu`

D — Menyembunyikan atribut harga agar tidak diubah sembarangan dari luar kelas

Jawaban: B

Interface cocok untuk hubungan “bisa” (*can-do*) antara kelas-kelas yang tidak bersaudara, persis seperti `DapatDicetak` di KedaiKita. Pilihan A adalah hubungan “adalah” dengan data bersama sehingga lebih cocok memakai kelas induk atau kelas abstrak. Pilihan C adalah konsep kelas dan objek biasa. Pilihan D adalah enkapsulasi.

Ringkasan

- **Abstraksi**: menonjolkan hal yang penting dan menyembunyikan detail yang tidak perlu.
- **Kelas abstrak** tidak bisa dibuat objeknya, tetapi boleh punya atribut, konstruktor, dan *method* konkret.
- *Method* abstrak **wajib** di-*override*. Pelanggaran ketahuan: Python saat objek dibuat, Java saat kompilasi.
- ***Template method***: *method* konkret yang menetapkan urutan langkah dan memanggil *method* abstrak.
- ***Interface***: kontrak kemampuan (*can-do*); satu kelas boleh memenuhi banyak *interface*.
- Java: `abstract class`, `interface`, `implements`. Python: `ABC` + `@abstractmethod`, atau `typing.Protocol`.
- KedaiKita: `Pembayaran` kini abstrak dengan `deskripsi()`; `Kedai` dan `Pesanan` sama-sama `DapatDicetak`.

Langkah Berikutnya

- Jalankan contoh di halaman praktik (Python 01—07, Java 01—06 dan 08) serta studi kasus KedaiKita.
- Kerjakan latihan, termasuk Latihan 5: Menu Abstrak, lalu cek pemahamanmu dengan kuis 10 soal.

Buka bahan pertemuan ini di GitHub:

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal

Pindai untuk membuka folder pertemuan ini di GitHub

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-10-Abstract-Class-dan-Interface